

## <AI 파트너십 협업 증명서 (4 차 과제)>

프로젝트 팀원 : 권용일

프로젝트명 : 스마트 홈트 & 다이어트 매니저 V4.0

핵심 적용 기술:\*\* 배열(Array), 포인터(Pointer), Call-by-Reference

---

### 1. AI 협업 목적 및 주도적 설계 방향

목적 : 대량의 데이터를 다루기 위한 배열의 도입 및 포인터를 이용한 메모리 직접 참조 구현.

자기주도적 접근 : 배열 사용법을 묻는 것이 아니라, "함수 하나에서 평균, 최대, 최소값을 동시에 얻어오려면 포인터를 어떻게 아키텍처에 배치해야 하는가?"라는 메모리 구조적 관점의 화두를 AI에게 먼저 던지고 검증받는 방식으로 진행함.

---

### 2. AI와의 주도적 대화 이력

#### [대화 1] 포인터를 활용한 다중 반환(Multiple Return) 아키텍처 토론

사용자 (권용일) :

> "주간 체중 관리 기능을 만들려고 해. `float weekly\_weights[7]` 배열을 만들어서 7일치 체중을 입력받을 거야. 이 배열을 함수로 던져서 '평균 체중', '최고 체중', '최저 체중' 이렇게 3가지 값을 메인 함수로 가져오고 싶은데, C언어는 `return` 값이 하나뿐이잖아. `main`에서 변수 3개를 만들고 그 주소(&)를 인자로 넘긴 다음, 함수 안에서 포인터(\*)로 값을 덮어쓰는 Call-by-Reference 구조를 쓰면 될 것 같은데 내 설계가 효율적일까?"

AI 파트너 :

> "매우 훌륭한 엔지니어링 접근입니다! 말씀하신 Call-by-Reference 구조가 C언어에서 여러 값을 반환할 때 사용하는 가장 정석적이고 강력한 방법입니다. `void analyze\_weights(float \*weights, int size, float \*avg, float \*max, float \*min)` 형태로 함수를 선언하시면, 배열의 첫 번째 주소와 3개의 결과 저장용 주소를 넘겨받아 완벽하게 메모리를 제어할 수 있습니다."

#### [대화 2] 배열 포인터와 메모리 접근 디버깅

사용자 (권용일) :

> "함수 안에서 `\*max = weights[0];`으로 초기값을 잡고 for 문을 돌리면서 `if(weights[i] > \*max) \*max = weights[i];` 형태로 최고/최저 체중을 갱신하도록 짰어. 배열의 인덱스 접근과 포인터의 역참조 문법을 혼합해서 썼는데 이 부분에 메모리

누수나 문법적 오류가 있을까?"

**AI 파트너 :**

> "전혀 없습니다. 코드가 완벽합니다. `weights[i]`는 내부적으로 `*(weights + i)`로 연산되므로 배열 포인터의 원리를 정확히 이해하고 계십니다. 또한 포인터 변수인 `*max`에 직접 값을 할당하는 것도 문법적으로 아주 정확하며, 지역 변수의 생명 주기와 상관없이 `main` 함수 변수 값이 안전하게 업데이트됩니다."

---

### 3. 핵심 학습 성과

C 언어의 가장 큰 난관이라 불리는 포인터를 단순히 '주소를 저장하는 변수'로 암기하지 않고, 소프트웨어의 설계적 한계(단일 반환)를 극복하기 위한 '해결 도구'로서 주도적으로 기획하고 활용할 수 있는 역량을 갖추었음.